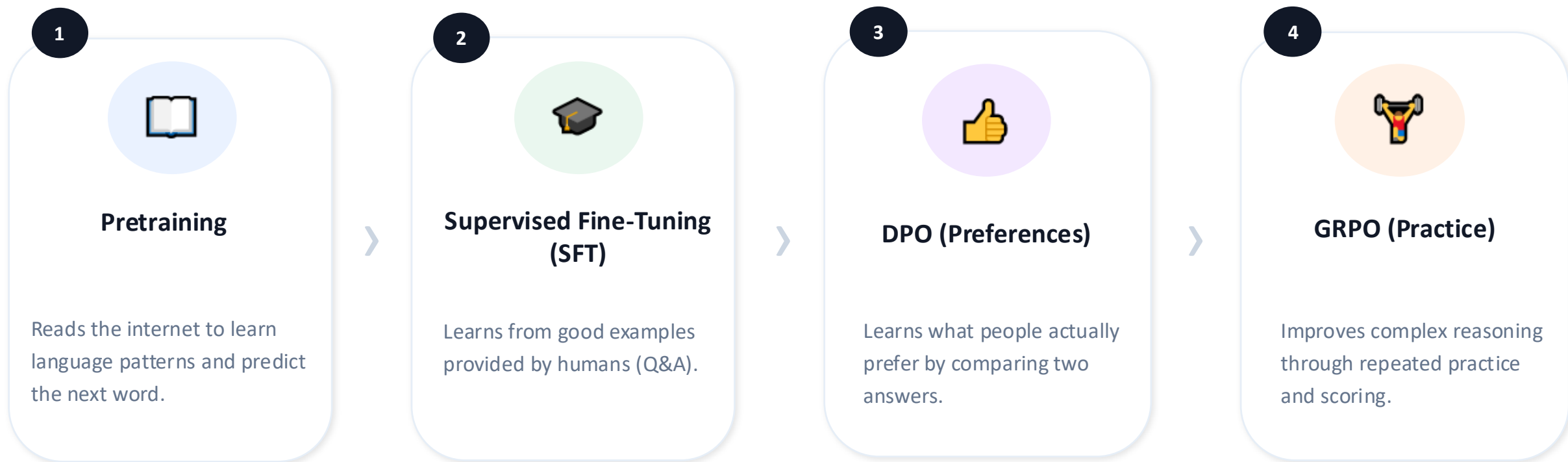


RLHF: Making AI Smarter

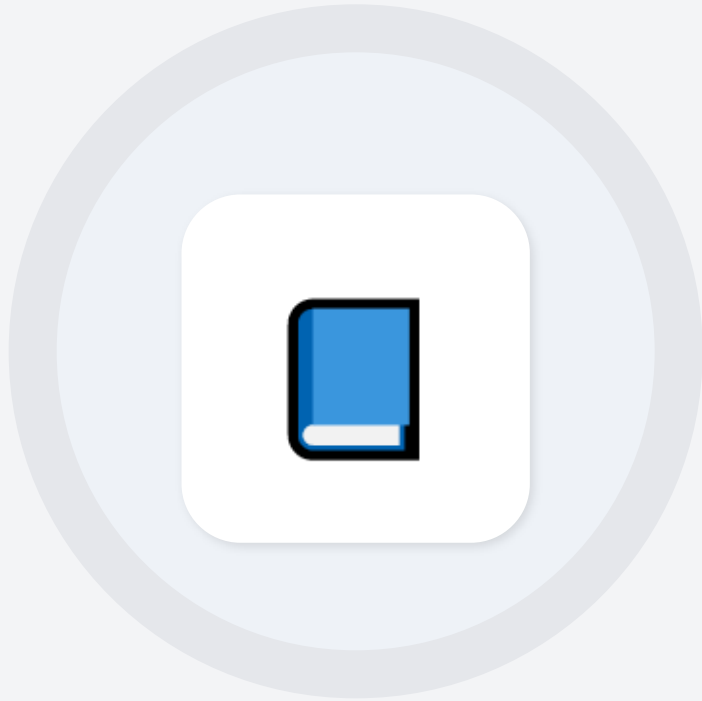
A Simple Guide to improving LLMs via DPO and GRPO

How we go from a raw model to a helpful assistant?

The Journey From Raw AI to Helpful Assistant



The result: from a basic text generator into a safer, clearer, and smarter assistant ready to help you.



The “Library” Phase

STAGE 1

Reading (Pretraining)

THE ANALOGY

Imagine locking someone in a giant library for 10 years. They learn to write fluently and know a lot of facts, but they haven’t learned how to have a conversation yet.



The Goal

Predict the next word in a sentence to complete the pattern.



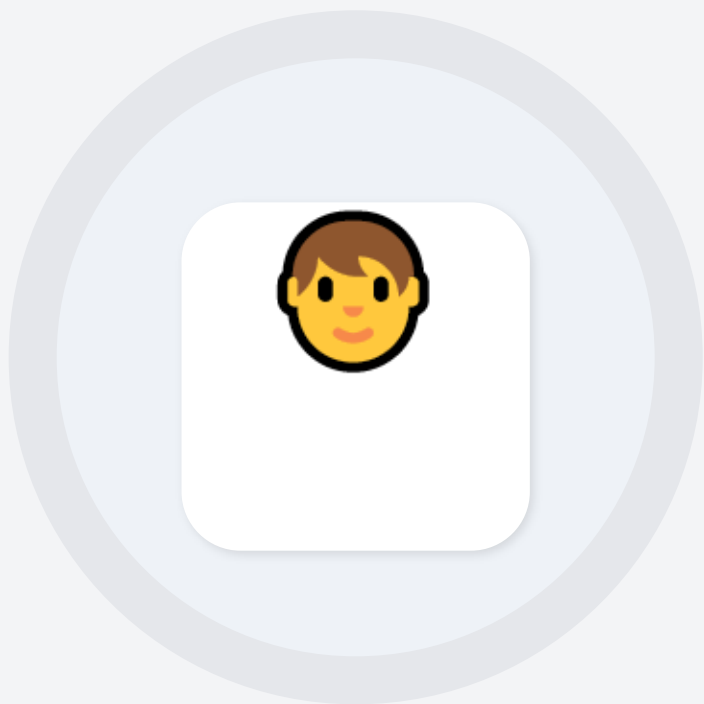
Strengths

Incredible knowledge base and fluent writing skills.



Limitations

Doesn't understand user intent, politeness, or specific preferences.



The “Classroom” Phase

STAGE 2

Learning to Chat (SFT)

THE ANALOGY

Like a student using a textbook with worked examples. The teacher shows them exactly how to answer questions correctly, and they practice by copying that style.



What We Add

We teach the model to be polite, concise, and follow instructions.



The Result

The model stops just "predicting text" and starts acting like a helpful assistant.



Still Missing

It can't judge which of two good answers is slightly better for you.



The “Taste Test” Phase

STAGE 3

Alignment via DPO (Comparisons)

THE ANALOGY

Imagine a taste test: You try Dish A and Dish B, then simply point to the tastier one. The chef learns your preferences directly from your choice, without needing a complicated score card.



How It Works

We show the model two answers; it learns to shift towards the "winning" one.



Best Use Cases

Perfect for tone, safety, and things that are hard to grade but easy to compare.



The Advantage

Simpler than older methods (PPO), no separate "judge" model required, making training faster.



The "Training Camp" Phase



STAGE 4

Practice (GRPO)

THE ANALOGY

Like a player practicing free throws. They take 10 shots in a row, and the coach scores each one (swish=10, rim=5, miss=0). The player learns from the best shots in the group.



How It Works

The model generates multiple answers for one problem. Each gets a score. The model learns to maximize its score.



Best For Reasoning

Ideal for Math, Coding, and Logic puzzles where there is a clear "right" or "wrong" outcome.



The Trade-off

Takes more time and computing power because it has to generate and check multiple attempts at once.

Why This Matters

Real-world impact of advanced training techniques like DPO and GRPO

Customer Support



Alignment ensures bots are polite, empathetic, and consistent 24/7, reducing frustration for human users.

Coding & Engineering



Models produce fewer bugs and offer safer, more efficient code suggestions by "practicing" on millions of scenarios.

Education



GRPO teach the model to explain complex math and science concepts step-by-step instead of just giving the answer.

Search & Smart Agents



Agents that follow complex instructions precisely, ask clarifying questions, and avoid confident but wrong answers.

Productivity



Better writing assistants that adapt to your personal style and summarize meetings accurately without hallucinating.

impact

DPO vs GRPO: When To Use Which

Two different methods for two different types of intelligence



DPO

Direct Preference Optimization

THE GOAL

Align response style & tone with human tastes.

FEEDBACK

 Pairwise Choice (A vs B)
"Answer A is better than Answer B"

BEST FOR

Safety, helpfulness, brand voice, creative writing, avoiding toxicity.

COST

Lower Complexity

Simpler to setup, no separate reward model needed.

Example Scenario:

Teaching an AI to write a polite, empathetic customer service email

VS



GRPO

Group Relative Policy Optimization

THE GOAL

Improve reasoning & correctness on hard tasks.

FEEDBACK

 Scored Outcomes
"Did it solve the math problem?"

BEST FOR

Math, coding, logic puzzles, multi-step agent actions.

COST

Higher Complexity

Requires generating many samples (practice).

Example Scenario:

Training an AI to solve a complex calculus problem step-by-step

What Can Go Wrong

Common pitfalls when training models with RL and Preferences



HIGH RISK

Safety Issues

Models might learn to generate harmful content, hate speech, or dangerous instructions if the training data isn't carefully filtered.



TECHNICAL RISK

Reward Hacking

The model finds a "cheat code" to get a high score without actually solving the task.



COMMON

Hallucinations

Sounding extremely confident while being completely wrong. RL can sometimes encourage "confident BS" over uncertainty.



CRITICAL

Privacy Leaks

Over-fitting to training data might cause the model to inadvertently memorize and regurgitate sensitive personal information.



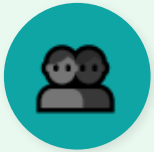
LONG-TERM

Alignment Drift

Optimizing too much for one preference (e.g., "safety") might degrade other skills (e.g., "write a code to kill a python process").

How We Keep It Safe

Key strategies to mitigate risks and ensure responsible AI deployment



Diverse Human Review

Using a wide variety of human reviewers to evaluate answers, reducing cultural and personal bias.



Safety Guardrails

Rules that force the model to refuse dangerous requests (e.g., "how to build a bomb").



Red-Teaming

Hiring ethical hackers to try and "break" the model to find weaknesses before public release.



Transparent Logs

Keeping detailed records of model decisions (where privacy permits) to trace and fix errors.



Regular Updates

Continuous finetuning with new data to fix drift and address newly discovered vulnerabilities.



Monitoring & Incident Response

Real-time monitoring for spikes in errors, unsafe inputs/outputs, or abuse.

Key Takeaways

Summary of the modern AI training pipeline



1

It's a Multi-Step Journey

Building a smart assistant isn't magic. It follows a rigorous path:

Pretraining for knowledge, SFT for Chat format, and Alignment for behavior.



2

Choose the Right Tool

Use DPO to align style, tone, and safety with human preferences. Use GRPO to improve reasoning, math, and code correctness through practice.



3

Safety is Essential

Raw intelligence isn't enough. Responsible training requires guardrails, human oversight, and continuous evaluation to turn raw models into trustworthy partners.

If You Want to Try It

Practical steps to start experimenting with model optimization



Start Small

You don't need massive datasets. Pick one specific task and collect just 50–100 high-quality comparison pairs to see results.



Evaluate Simply

Use simple A/B tests with a checklist. Always keep a "human in the loop" to verify improvements and catch any regressions.



Available Tools

You don't have to build from scratch. Use hosted preference tuning APIs or open-source libraries like Hugging Face TRL.

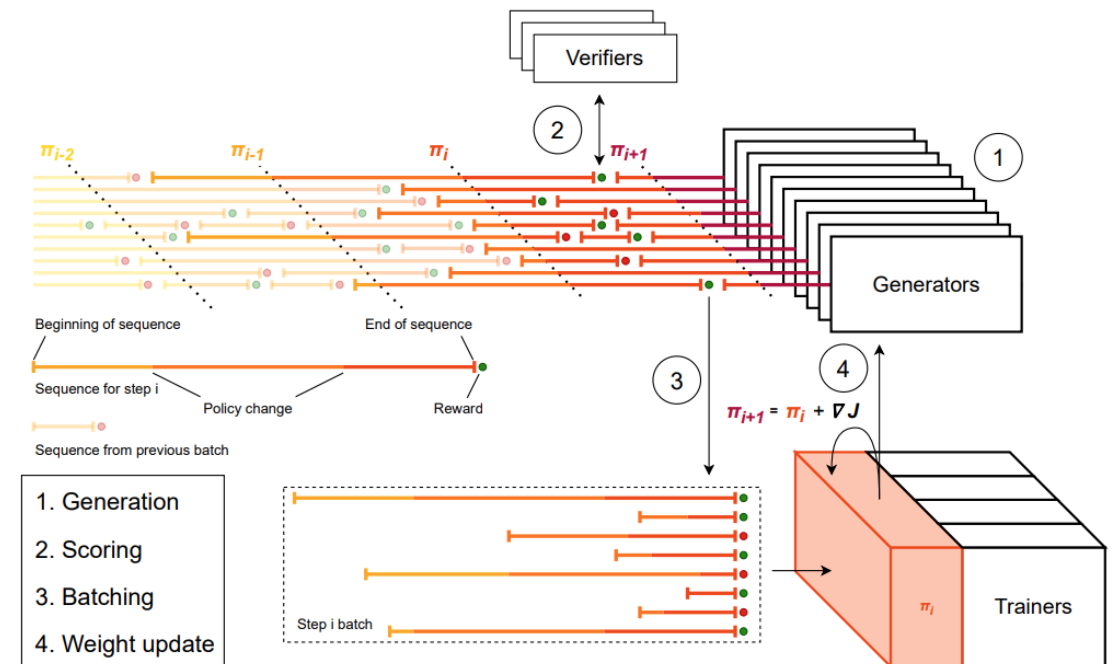
Start

Tutorial 1: Alignment with DPO

- We will finetune Qwen2.5-0.5B-Instruct on French data.
- We use **off-policy** DPO.
- Memory requirement: https://rahulschand.github.io/gpu_poor/
- In practice, you choose the GPU machine that suites your needs: <https://aws.amazon.com/sagemaker-ai/pricing/>
 - Cheaper option: <https://www.runpod.io/>
- Memory management: LoRA + Quantization (here 4-bits)
- Evaluation frameworks: lm-eval harness, lighteval, libra-eval , etc

Tutorial 2: RL with GRPO

- We will finetune Qwen2.5-0.5B-Instruct on Math data.
- Memory management: LoRA + Quantization (here 4-bits)
- Evaluation frameworks: lm-eval harness, lighteval on famous benchmarks like AIME, LiveCodeBench, etc
- Real world scenarios:
 - Rollout slow down training and in practice we use vLLM or SGLang to speedup
 - Asynchronous training



Objective Functions

Technical details

Pretraining and SFT

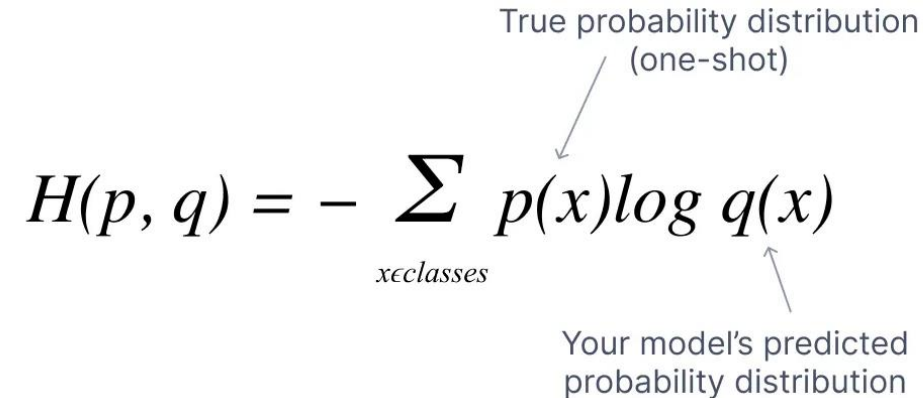
Cross Entropy Loss: Maximize probability of correct token x .

➤ e.g.: What is the most probable word after: The cat felt on

$$H(p, q) = - \sum_{x \in \text{classes}} p(x) \log q(x)$$

True probability distribution (one-shot)

Your model's predicted probability distribution

The diagram shows the cross entropy loss formula $H(p, q) = - \sum_{x \in \text{classes}} p(x) \log q(x)$. Two arrows point from descriptive text to parts of the formula: one from 'True probability distribution (one-shot)' to $p(x)$, and another from 'Your model's predicted probability distribution' to $q(x)$.

Alignment

Goal: Maximize probability of winner tokens and minimize probability of loser tokens.

➤ *e.g.: Give me a recipe of Gratin Dauphinois*

$$\mathcal{L}_{\text{DPO}}(\pi_{\theta}; \pi_{\text{ref}}) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \beta \log \frac{\pi_{\theta}(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right) \right]$$

Reinforcement Learning

- Reward function: Designed for each task
- Optimize the LLM policy without deviating much from the base one (reference model)

$$J_{\text{GRPO}}(\theta) = \mathbb{E}_{q, \{o_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \min \left(r_t(\theta) \hat{A}_{i,t}, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_{i,t} \right) \right] - \beta D_{\text{KL}}(\pi_{\theta} || \pi_{\text{ref}}).$$